

# Sistema de Backup en C

Este proyecto implementa un sistema de respaldo de archivos en C que compara el rendimiento de dos métodos de copia: uno usando syscalls POSIX directas (open/read/write), y otro usando la biblioteca estándar C (fopen/fread/fwrite). Incluye manejo de errores con errno, medición de tiempo con time.h, sistema de log y aviso cuando un archivo no fue modificado desde el último backup.

## 1. Descripción del proyecto

El sistema está dividido en tres archivos. **smart\_copy.h** define los tipos y constantes compartidos. **backup\_engine.c** implementa `sys_smart_copy()`, la función principal que usa syscalls POSIX directamente para leer y escribir archivos sin ninguna capa intermedia. **main.c** implementa `stdio_copy()`, el método de referencia con `fread/fwrite`, y el benchmark que compara los dos.

| Archivo         | Función principal             | Método de I/O                   |
|-----------------|-------------------------------|---------------------------------|
| smart_copy.h    | Tipos, constantes, prototipos | —                               |
| backup_engine.c | sys_smart_copy()              | open / read / write / close     |
| main.c          | stdio_copy() + benchmark      | fopen / fread / fwrite / fclose |

```
backup/
■■■ smart_copy.h      # Interfaz pública: tipos, constantes, prototipos
■■■ backup_engine.c   # Motor de copia con syscalls POSIX directas
■■■ main.c            # Benchmark, stdio_copy y punto de entrada
■■■ backup            # Ejecutable compilado
■■■ output/          # Archivos generados (se crea automáticamente)
    ■■■ backup.log    # Log de todas las operaciones
    ■■■ *.smart       # Copias hechas con sys_smart_copy
    ■■■ *.stdio       # Copias hechas con stdio_copy
```

## 2. Implementación

### **sys\_smart\_copy()** — syscalls POSIX directas

Esta función abre el archivo con `open()`, lee bloques de **4096 bytes** con `read()` y los escribe con `write()`. El tamaño de 4096 bytes coincide con el tamaño de página de la MMU del procesador, lo que hace la transferencia más eficiente porque el kernel puede alinear las operaciones con las páginas físicas de memoria. Incluye un bucle interno para manejar *short-writes* y usa `switch(errno)` para distinguir tres tipos de error específicos.

| Paso | Syscall               | Qué hace                                                                               |
|------|-----------------------|----------------------------------------------------------------------------------------|
| 1    | <code>access()</code> | Verifica permisos antes de abrir el archivo                                            |
| 2    | <code>open()</code>   | Abre origen ( <code>O_RDONLY</code> ) y crea destino ( <code>O_WRONLY O_CREAT</code> ) |
| 3    | <code>read()</code>   | Lee hasta 4096 bytes del origen al buffer                                              |
| 4    | <code>write()</code>  | Escribe esos bytes al destino — con bucle para short-writes                            |
| 5    | <code>close()</code>  | Cierra ambos descriptores en la sección cleanup                                        |

### **Medición de tiempo con time.h**

El programa mide el tiempo de ejecución de ambos métodos usando `clock_gettime(CLOCK_MONOTONIC)` de `time.h`. Se usan dos "fotos" del reloj — una antes y una después del bucle de copia — y se calcula la diferencia con precisión de nanosegundos. `CLOCK_MONOTONIC` garantiza que el reloj nunca retrocede aunque el usuario cambie la hora del sistema.

```
#include

struct timespec t_start, t_end;
clock_gettime(CLOCK_MONOTONIC, &t_start); // antes
// ... bucle read/write ...
clock_gettime(CLOCK_MONOTONIC, &t_end);    // después

elapsed = (t_end.tv_sec - t_start.tv_sec)
          + (t_end.tv_nsec - t_start.tv_nsec) * 1e-9;
```

### **stdio\_copy()** — biblioteca estándar C

Esta función usa `fopen()`, `fread()` y `fwrite()`. `fread()` acumula datos en un buffer interno de ~8KB gestionado por la libc antes de pasarlos al programa. El buffer de trabajo se mantiene en 4096 bytes igual que `sys_smart_copy` para que la comparación sea justa — la única variable que cambia es el método.

| Aspecto | <code>sys_smart_copy</code> | <code>stdio_copy</code> |
|---------|-----------------------------|-------------------------|
|---------|-----------------------------|-------------------------|

|               |                         |                                 |
|---------------|-------------------------|---------------------------------|
| Abre con      | open() → int fd         | fopen() → FILE*                 |
| Lee con       | read() — directo al SO  | fread() — pasa por buffer libc  |
| Escribe con   | write() — directo al SO | fwrite() — pasa por buffer libc |
| Buffer propio | char buffer[4096]       | char buffer[4096]               |
| Buffer extra  | ninguno                 | ~8KB interno de libc            |
| Copias en RAM | 1 (disco → buffer)      | 2 (disco → libc → buffer)       |

### Manejo de errores con switch(errno)

Ambos métodos usan switch(errno) para distinguir tres tipos de error en lugar de un perror() genérico. El enunciado pide gestionar archivos inexistentes, falta de permisos y disco lleno — los tres están cubiertos:

| errno  | Código propio     | Mensaje al usuario           | Cuándo ocurre                     |
|--------|-------------------|------------------------------|-----------------------------------|
| ENOENT | SC_ERR_NOENT (-6) | [ERROR] El archivo no existe | Ruta incorrecta o archivo borrado |
| EACCES | SC_ERR_PERM (-5)  | [ERROR] Permiso denegado     | Archivo sin permisos de lectura   |
| ENOSPC | SC_ERR_NOSPC (-7) | [ERROR] Disco lleno          | Sin espacio al escribir           |

### 3. Resultados del benchmark

Mac OS — Apple M1 — SSD NVMe — 06/04/2026

| Tamaño | sys_smart_copy | Velocidad   | stdio_copy | Velocidad   | Ganador   |
|--------|----------------|-------------|------------|-------------|-----------|
| 1 KB   | 0.000075 s     | 13.02 MB/s  | 0.000147 s | 6.64 MB/s   | syscall ✓ |
| 1 MB   | 0.002489 s     | 401.77 MB/s | 0.002370 s | 421.94 MB/s | fread ✓   |
| 1 GB   | 1.488406 s     | 687.98 MB/s | 1.424912 s | 718.64 MB/s | fread ✓   |

```
laura@Lauras-MacBook-Air backup % ./backup --benchmark

BENCHMARK: sys_smart_copy vs stdio_copy
buffer = 4096 bytes (PAGE_SIZE)

Tamaño  sys_smart_copy      stdio_copy      Ganador
-----  -
Generando 1 KB...
[OK] stdio_copy: 1024 bytes copiados
1 KB    0.000075s    13.02 MB/s    0.000147s    6.64 MB/s    syscall ✓
Generando 1 MB...
[OK] stdio_copy: 1048576 bytes copiados
1 MB    0.002489s    401.77 MB/s    0.002370s    421.94 MB/s    fread ✓
Generando 1 GB...
[OK] stdio_copy: 1073741824 bytes copiados
1 GB    1.488406s    687.98 MB/s    1.424912s    718.64 MB/s    fread ✓

Análisis:
1 KB → fread gana: buffer interno libc absorbe el
        archivo en 1 solo context switch al kernel.
1 GB → syscall gana: fread hace copia extra en RAM
        (disco→buf_libc→tu_buf) en cada iteración.
```

Figura 1 — Salida real del comando ./backup --benchmark

#### Análisis — por qué fread es más eficiente en archivos pequeños

La comparación no es solo entre dos funciones — es entre **una copia en RAM vs dos copias en RAM**. `fread()` tiene un buffer interno de ~8KB gestionado por la libc en espacio de usuario. Cuando el archivo cabe en ese buffer, el kernel se invoca una sola vez sin importar cuántas veces el programa llame a `fread()`. Esto reduce el número de context switches.

#### 1 KB — syscall gana (13.02 vs 6.64 MB/s)

Con archivos de 1KB el overhead de inicializar el buffer interno de `fread()` supera su ventaja. `sys_smart_copy` va directo al kernel con una sola llamada a `read()` y termina antes. Este resultado muestra que el buffering en espacio de usuario no siempre ayuda — depende del tamaño del archivo.

#### 1 MB — fread gana por poco (421.94 vs 401.77 MB/s)

Con 1MB los dos métodos son casi equivalentes. Es la zona de equilibrio donde el caché del sistema operativo nivela la diferencia. El buffer interno de `fread()` empieza a reducir los context switches lo suficiente para superar a `syscall`.

### **1 GB — fread gana (718.64 vs 687.98 MB/s)**

En archivos grandes el buffer interno de `fread()` reduce significativamente las interrupciones al kernel. Sin embargo, `fread()` hace dos copias en RAM por cada chunk (disco → buffer libc → buffer propio), mientras que `read()` hace solo una (disco → buffer propio). En un disco duro convencional, donde la velocidad de lectura es menor, la ventaja de `syscalls` directas sería más visible porque la copia extra en RAM costaría más en proporción.

## 4. Funciones adicionales

### Sistema de log — output/backup.log

Cada operación queda registrada con timestamp, método, rutas, bytes transferidos, tiempo y throughput. Usa modo append para no borrar entradas anteriores.

```
[2026-04-06 19:17:05] syscall | bk_src.bin → bk_dst_smart.bin | 1024 bytes | 0.000075s  
| 13.02 MB/s | OK  
[2026-04-06 19:17:05] stdio | bk_src.bin → bk_dst_stdio.bin | 1024 bytes | 0.000147s  
| 6.64 MB/s | OK
```

```
output > ≡ backup.log  
1 [2026-04-06 07:45:25] syscall | test.txt → output/test_backup.txt.smart | 12 bytes | 0.000351s | 0.03 MB/s | OK  
2 [2026-04-06 07:45:25] stdio | test.txt → output/test_backup.txt.stdio | 12 bytes | 0.000158s | 0.07 MB/s | OK  
3 [2026-04-06 19:17:05] syscall | output/bk_src.bin → output/bk_dst_smart.bin | 1024 bytes | 0.000075s | 13.02 MB/s | OK  
4 [2026-04-06 19:17:05] stdio | output/bk_src.bin → output/bk_dst_stdio.bin | 1024 bytes | 0.000147s | 6.64 MB/s | OK  
5 [2026-04-06 19:17:05] syscall | output/bk_src.bin → output/bk_dst_smart.bin | 1048576 bytes | 0.002489s | 401.77 MB/s | OK  
6 [2026-04-06 19:17:05] stdio | output/bk_src.bin → output/bk_dst_stdio.bin | 1048576 bytes | 0.002370s | 421.94 MB/s | OK  
7 [2026-04-06 19:17:15] syscall | output/bk_src.bin → output/bk_dst_smart.bin | 1073741824 bytes | 1.488406s | 687.98 MB/s | OK  
8 [2026-04-06 19:17:17] stdio | output/bk_src.bin → output/bk_dst_stdio.bin | 1073741824 bytes | 1.424912s | 718.64 MB/s | OK  
9 [2026-04-06 19:25:16] syscall | test.txt → output/test_backup.txt.smart | 12 bytes | 0.000570s | 0.02 MB/s | OK  
10 [2026-04-06 19:25:16] stdio | test.txt → output/test_backup.txt.stdio | 12 bytes | 0.000336s | 0.03 MB/s | OK  
11
```

Figura 2 — Contenido real de output/backup.log

### Historial de backups — --check

Busca en el log todas las entradas de un archivo y muestra su historial completo con fechas y resultados.

```
./backup --check test.txt
```

```
laura@Lauras-MacBook-Air backup % ./backup --check test_backup.txt  
  
Historial de backups para: test_backup.txt  
Fecha/hora      Método Destino      Bytes Estado  
-----  
2026-04-06 07:45:25  syscall 0 output/test_backup.txt.smart 12 OK  
2026-04-06 07:45:25  stdio 0 output/test_backup.txt.stdio 12 OK  
2026-04-06 19:25:16  syscall 0 output/test_backup.txt.smart 12 OK  
2026-04-06 19:25:16  stdio 0 output/test_backup.txt.stdio 12 OK  
  
Total: 4 backup(s) encontrado(s) para 'test_backup.txt'
```

Figura 3 — Historial de backups con --check

### Aviso de archivo no modificado

Antes de hacer un backup, compara la fecha de modificación del archivo (mtime via stat()) con la fecha del último backup en el log. Si no fue modificado, avisa y pide confirmación:

```
--- Respaldando 'test.txt' ---  
  
[WARNING] El archivo 'test.txt' no fue modificado  
desde el último backup registrado.  
¿Deseas hacer el backup de todos modos? [s/N]: s
```

```
sys_smart_copy (syscall)    0.000570 s    0.02 MB/s    2 ops
stdio_copy      (fread)     0.000336 s    0.03 MB/s    2 ops
Ganador: stdio_copy
```

```
laura@Lauras-MacBook-Air backup % ./backup -b test.txt test_backup.txt
```

```
--- Respalando 'test.txt' ---
```

```
[WARNING] El archivo 'test.txt' no fue modificado
          desde el último backup registrado.
          ¿Deseas hacer el backup de todos modos? [s/N]: n
[INFO] Backup cancelado.
```

```
laura@Lauras-MacBook-Air backup % ./backup -b test.txt test_backup.txt
```

```
--- Respalando 'test.txt' ---
```

```
[WARNING] El archivo 'test.txt' no fue modificado
          desde el último backup registrado.
          ¿Deseas hacer el backup de todos modos? [s/N]: s
```

```
[OK] stdio_copy: 12 bytes copiados
```

Resultados:

| Método                   | Tiempo     | Throughput | Ops   |
|--------------------------|------------|------------|-------|
| sys_smart_copy (syscall) | 0.000570 s | 0.02 MB/s  | 2 ops |
| stdio_copy (fread)       | 0.000336 s | 0.03 MB/s  | 2 ops |

Ganador: stdio\_copy

Figura 3 — Warning de archivo sin cambios y confirmación del usuario

## 5. Conclusiones

---

Los resultados demuestran que no existe un método universalmente superior. Con archivos pequeños (1KB), las syscalls directas son más rápidas porque evitan el overhead del buffer interno de la libc. Con archivos grandes (1MB+), `fread()` es más eficiente porque su buffer en espacio de usuario reduce el número de context switches al kernel.

| Escenario              | Método más rápido           | Razón principal                   |
|------------------------|-----------------------------|-----------------------------------|
| Archivos < 8KB         | <code>sys_smart_copy</code> | Sin overhead del buffer libc      |
| Archivos entre 1-100MB | <code>stdio_copy</code>     | Buffer reduce context switches    |
| Archivos > 1GB (SSD)   | Muy similar                 | SSD rápido minimiza la diferencia |
| Archivos > 1GB (HDD)   | <code>sys_smart_copy</code> | Evita la copia extra en RAM       |

El concepto más importante del proyecto es que el número de veces que el programa interrumpe al sistema operativo (context switches) afecta directamente el rendimiento. Por eso se usa un buffer de **4096 bytes (PAGE\_SIZE)**: para hacer menos llamadas al kernel y transferir más datos en cada una. Según el documento de referencia del curso, usar buffers de 4KB en lugar de escribir byte a byte puede ser hasta **1584 veces más rápido**.